E9: 309 ADL 18-11-2020

http://leap.ee.iisc.ac.in/sriram/teaching/ADL2020/

Housekeeping

Attachments [Deadline : Nov 19: 10 AM]

Firstname_Lastname_proj1_slides.ppt

Firstname_Lastname_proj1_report.pdf

Teams assignment link.

- 1st mini-project
  - ✓ Deadlines
    - ★ Presentation on Nov19 and Nov20
      - ★ Your date allocation has been finalized
      - ★ Presentation and report template will be sent out this week.
        - ★ Report 1 page + references and tools
        - ★ Slides 4 slides for individual project and 6 slides for 2-member.

Recap of previous class

Representation learning/data-visualization

- ✴️ Restricted Boltzmann machine
  - ➡️ Energy based model
    - ✓ Conditional independence
    - ✓ Sigmoidal function for conditional probability $p(h_j = 1 / v) = \sigma(\cdot)$
  - ➡️ Issues in RBM training

RBM - Training

- Model parameters $\Theta = \{\mathbf{W}, \mathbf{b}, \mathbf{c}\}$ $\quad \mathbf{x} = [\mathbf{v}^T \ \mathbf{h}^T]^T$
  - ✓ Learnt by maximizing the log-likelihood $log(p(\mathbf{x};\Theta))$
  - ✓ Non-convex optimization.

as cent

- Gradient ~~descent~~ based optimization

un-normalized

$$\frac{\partial log(p(\mathbf{x};\Theta))}{\partial \Theta} = \frac{\partial log(\tilde{p}(\mathbf{x};\Theta))}{\partial \Theta} - \frac{\partial log(Z(\Theta))}{\partial \Theta}$$

$$\frac{\partial log(Z(\Theta))}{\partial \Theta} = \frac{1}{Z}\frac{\partial Z(\Theta)}{\partial \Theta}$$

$$Z(\Theta) = \sum_{\mathbf{x}} \tilde{p}(\mathbf{x};\Theta))$$

RBM - Training

- For exponential families

$$\tilde{p}(\mathbf{x};\Theta) > 0 \quad \forall\ \mathbf{x}$$

$$\frac{\partial log(Z(\Theta))}{\partial \Theta} = \frac{1}{Z}\sum_{\mathbf{x}} \frac{\partial \tilde{p}(\mathbf{x};\Theta)}{\partial \Theta}$$

$$\frac{\partial log(Z(\Theta))}{\partial \Theta} = \frac{1}{Z}\sum_{\mathbf{x}} \frac{\partial exp(log(\tilde{p}(\mathbf{x};\Theta)))}{\partial \Theta}$$

$$= \frac{1}{Z}\sum_{\mathbf{x}} exp(log(\tilde{p}(\mathbf{x};\Theta)))\frac{\partial (log(\tilde{p}(\mathbf{x};\Theta)))}{\partial \Theta}$$

$$= \sum_{\mathbf{x}} p(\mathbf{x};\Theta)\frac{\partial (log(\tilde{p}(\mathbf{x};\Theta)))}{\partial \Theta}$$

RBM - Training

Intractable

$$\boxed{\frac{\partial(logZ(\Theta))}{\partial\Theta} = \mathbb{E}_{\mathbf{x}\sim p}\left[\frac{\partial\tilde{p}(\mathbf{x},\Theta)}{\partial\Theta}\right]}$$

- ✴️ Intractable to compute the exact gradient of the negative phase
  - ➡️ Using approximations to gradients
    - ✓ Based on sampling methods.
      - ★ Monte-carlo Markov Chain (MCMC) based approximation
      - ★ Resorting to Gibbs sampling.

Approximating expectations

- Expectation is intractable in the gradient computation.

$$\mathbf{s} = \mathbb{E}_{x\sim p}[f(\mathbf{x})]$$

$$\hat{\mathbf{s}}_n = \frac{1}{n}\sum_{i=1}^{n} f(\mathbf{x}_i)$$

$f(x) = \frac{\partial}{\partial \theta} \log \tilde{p}(x;\theta)$

$\hat{s}_n \to s$

- Using law of large numbers and central limit theorem

$$\mathbb{E}[\hat{\mathbf{s}}_n] = \mathbf{s} \; ; \quad \lim_{n\to\infty} \hat{\mathbf{s}}_n \to \mathbf{s} \; ; \; Var[\hat{\mathbf{s}}_n] = \frac{Var[f(\mathbf{x})]}{n}$$

x_i ?

every step of gradient ascent

$\theta = \{w, b, c\} \longrightarrow p(x;\theta)$

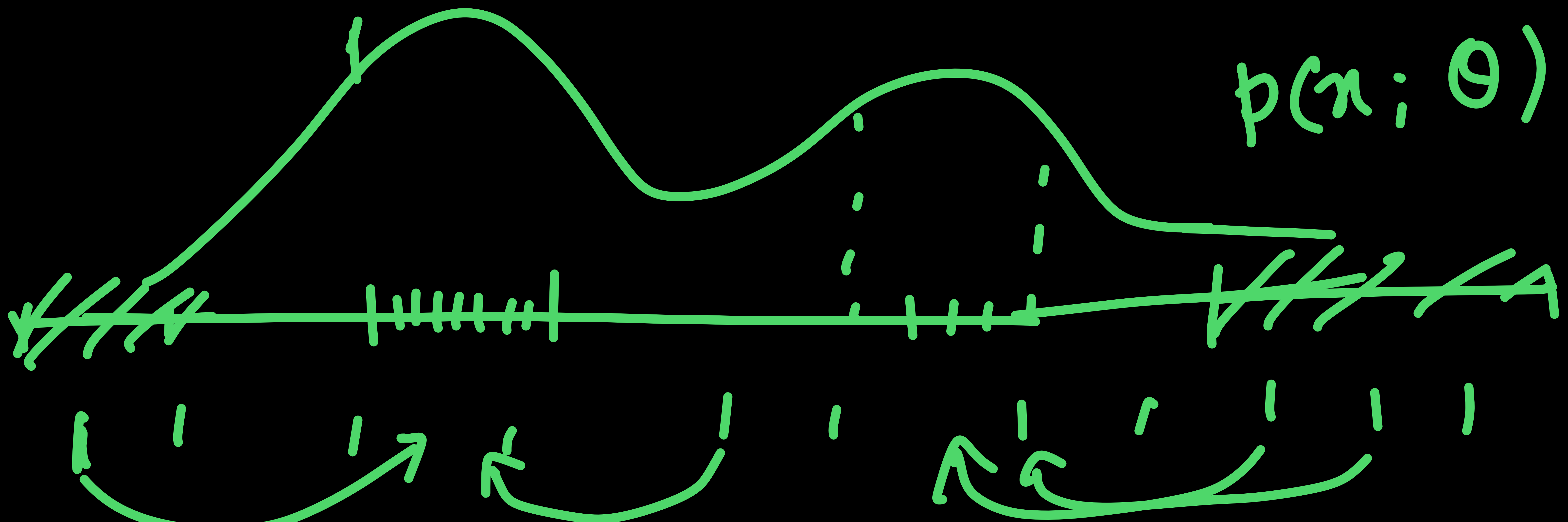
gradient ascent

sampling

$\{x_i\}_{i=1}^{n}$

gradient approx.

negative phase



$p(x;\theta)$

Sampling

- The question of finding the right samples $\mathbf{x}_i$
- Should we sample based on the p or some other function q

$$p(\mathbf{x})f(\mathbf{x}) = q(\mathbf{x})\frac{p(\mathbf{x})f(\mathbf{x})}{q(\mathbf{x})}$$

- Using a suitable function q, the estimate of the intractable expectation is

$\hat{s}_q$

$$\hat{\mathbf{s}}_q = \frac{1}{n}\sum_{i=1,\mathbf{x}_i \sim q}^{n} \frac{p(\mathbf{x}_i)f(\mathbf{x}_i)}{q(\mathbf{x}_i)}$$

Infeasible
RBMs

Importance Sampling

Gibbs sampling with random initialization

- Using Markov chain Monte-Carlo (MCMC) sampling $p_{model}(\mathbf{x})$
  - ✓ Initialize random samples using uniform distribution.
  - ✓ Use the data samples to perform new updates of the samples
  - ✓ Perform k steps of this update.

$\tilde{x} \leftarrow p(x;\theta)$

$\tilde{x}^0 \rightarrow \tilde{x}^1 \rightarrow \tilde{x}^2 \rightarrow \cdots$

k-steps

Markov approx.

Gibbs sampling with random initialization

Algorithm 18.1 A naive MCMC algorithm for maximizing the log-likelihood with an intractable partition function using gradient ascent.

Set ϵ, the step size, to a small positive number.
Set k, the number of Gibbs steps, high enough to allow burn in. Perhaps 100 to train an RBM on a small image patch.
while not converged **do**
  Sample a minibatch of m examples $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$ from the training set.
  $\boxed{\mathbf{g} \leftarrow \frac{1}{m}\sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} \log \tilde{p}(\mathbf{x}^{(i)}; \boldsymbol{\theta})}$. positive phase
  Initialize a set of m samples $\{\tilde{\mathbf{x}}^{(1)}, \ldots, \tilde{\mathbf{x}}^{(m)}\}$ to random values (e.g., from a uniform or normal distribution, or possibly a distribution with marginals matched to the model's marginals).
  for $i = 1$ to k **do**
    for $j = 1$ to m **do**
      $\tilde{\mathbf{x}}^{(j)} \leftarrow \text{gibbs\_update}(\tilde{\mathbf{x}}^{(j)})$.
    end for
  end for
  $\mathbf{g} \leftarrow \mathbf{g} - \frac{1}{m}\sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} \log \tilde{p}(\tilde{\mathbf{x}}^{(i)}; \boldsymbol{\theta})$. negative phase
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \epsilon \mathbf{g}$.
end while

Chap 18, 20 of Deep Learning Book

Gibbs sampling

Gibbs sampling with random initialization

- Using Markov chain Monte-Carlo (MCMC) sampling $\boxed{p_{model}(\mathbf{x})}$

  ✓ Use the given data samples and the current model weights to perform Gibbs sampling

  $x_1 \ldots x_N$

  $\downarrow$

  neighborhood

  $\tilde{x}_1 \ldots \tilde{x}_N$

  ★ Initially the model weights make for a poor estimation of the negative phase of the gradient.

  ★ But the positive phase makes up for the lossy estimate.

  ★ Once the model weights are updated for a few iterations

  ★ The negative phase becomes more accurate.

$\boxed{\text{Contrastive Divergence}}$

Gibbs sampling with random initialization

Algorithm 18.2 The contrastive divergence algorithm, using gradient ascent as the optimization procedure.

Set ϵ, the step size, to a small positive number.
Set k, the number of Gibbs steps, high enough to allow a Markov chain sampling from $p(\mathbf{x};\boldsymbol{\theta})$ to mix when initialized from p_{data}. Perhaps 1-20 to train an RBM on a small image patch.
while not converged **do**
  Sample a minibatch of m examples $\underline{\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}}$ from the training set.
  $\mathbf{g} \leftarrow \frac{1}{m}\sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} \log \tilde{p}(\mathbf{x}^{(i)};\boldsymbol{\theta})$. ← + phase
  for $i = 1$ to m **do**
    $\underline{\tilde{\mathbf{x}}^{(i)}} \leftarrow \underline{\mathbf{x}^{(i)}}$.
  end for
  for $i = 1$ to k **do**
    for $j = 1$ to m **do**
      $\tilde{\mathbf{x}}^{(j)} \leftarrow \underline{\text{gibbs\_update}(\tilde{\mathbf{x}}^{(j)})}$.
    end for
  end for
  $\mathbf{g} \leftarrow \mathbf{g} - \frac{1}{m}\sum_{i=1}^{m} \underline{\nabla_{\boldsymbol{\theta}} \log \tilde{p}(\tilde{\mathbf{x}}^{(i)};\boldsymbol{\theta})}$. ← -ve phase
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \epsilon \mathbf{g}$.
end while

Contrastive divergence
k-step.

One step contrastive divergence - Training example

$$\boxed{\mathbf{x} = [\mathbf{v}^T \ \mathbf{h}^T]^T}$$

- Given a set of visible data $\{\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_N\} \in \mathcal{B}^D$
- Randomly initialize the model parameters $\Theta^0 = \{\mathbf{W}^0, \mathbf{b}^0, \mathbf{c}^0\}$; $k = 0$ iteration
  - ✓ Sampling $\{\mathbf{h}_1, \mathbf{h}_2, ..., \mathbf{h}_N\} \in \mathcal{B}^d$
    - ◉ Using conditional independence of hidden given visible
      - ★ Sample each $\{\tilde{\mathbf{h}}_1, \tilde{\mathbf{h}}_2, ..., \tilde{\mathbf{h}}_N\} \in \mathcal{B}^d$

$\tilde{h}_q$

$$p(h_{q,j} = 1|\mathbf{v}_q) = \sigma(\mathbf{v}_q^T \mathbf{W}^k_{:,j} + c^k_j) \quad q = \{1, .., N\}, j = \{1, .., d\}$$

One step contrastive divergence - Training example

$$\mathbf{x} = [\mathbf{v}^T \ \mathbf{h}^T]^T$$

- Given a set of visible data $\{\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_N\} \in \mathcal{B}^D$
- Randomly initialize the model parameters $\Theta^0 = \{\mathbf{W}^0, \mathbf{b}^0, \mathbf{c}^0\}$ $k = 0$
  - ✓ Sampling visible nodes again using $[\tilde{h}_1, \tilde{h}_2, \dots \tilde{h}_N]$
    - ◉ Using conditional independence of visible given hidden
      - ★ Sample visible nodes $\{\tilde{\mathbf{v}}_1, \tilde{\mathbf{v}}_2, ..., \tilde{\mathbf{v}}_N\} \in \mathcal{B}^D$

$$\begin{bmatrix} \tilde{v}_1 & \tilde{v}_2 & \dots \tilde{v}_N \\ \tilde{h}_1 & \tilde{h}_2 & \tilde{h}_N \end{bmatrix}$$

$\tilde{x}_1 \quad \tilde{x}_2 \dots \tilde{x}_N$

$$p(v_{q,i} = 1|\tilde{\mathbf{h}}_q) = \sigma(\mathbf{W}^k_{i,:}\tilde{\mathbf{h}}^T_q + b^k_i) \quad q = \{1, .., N\}, i = \{1, .., D\}$$

One-step contrastive divergence

log $\tilde{p}(x; \theta)$

- Computing the gradient

Observed data

positive phase

$- E[v, h]$

$v^T W h$

$$\frac{\partial(p([\mathbf{v}\ \mathbf{h}], \Theta)}{\partial \mathbf{W}} \approx \frac{1}{N}\sum_{q=1}^{N} \mathbf{v}_q \mathbf{h}_q^T - \frac{1}{N}\sum_{q=1}^{N} \tilde{\mathbf{v}}_q \tilde{\mathbf{h}}_q^T$$

log $\tilde{p}(\tilde{x}; \theta)$

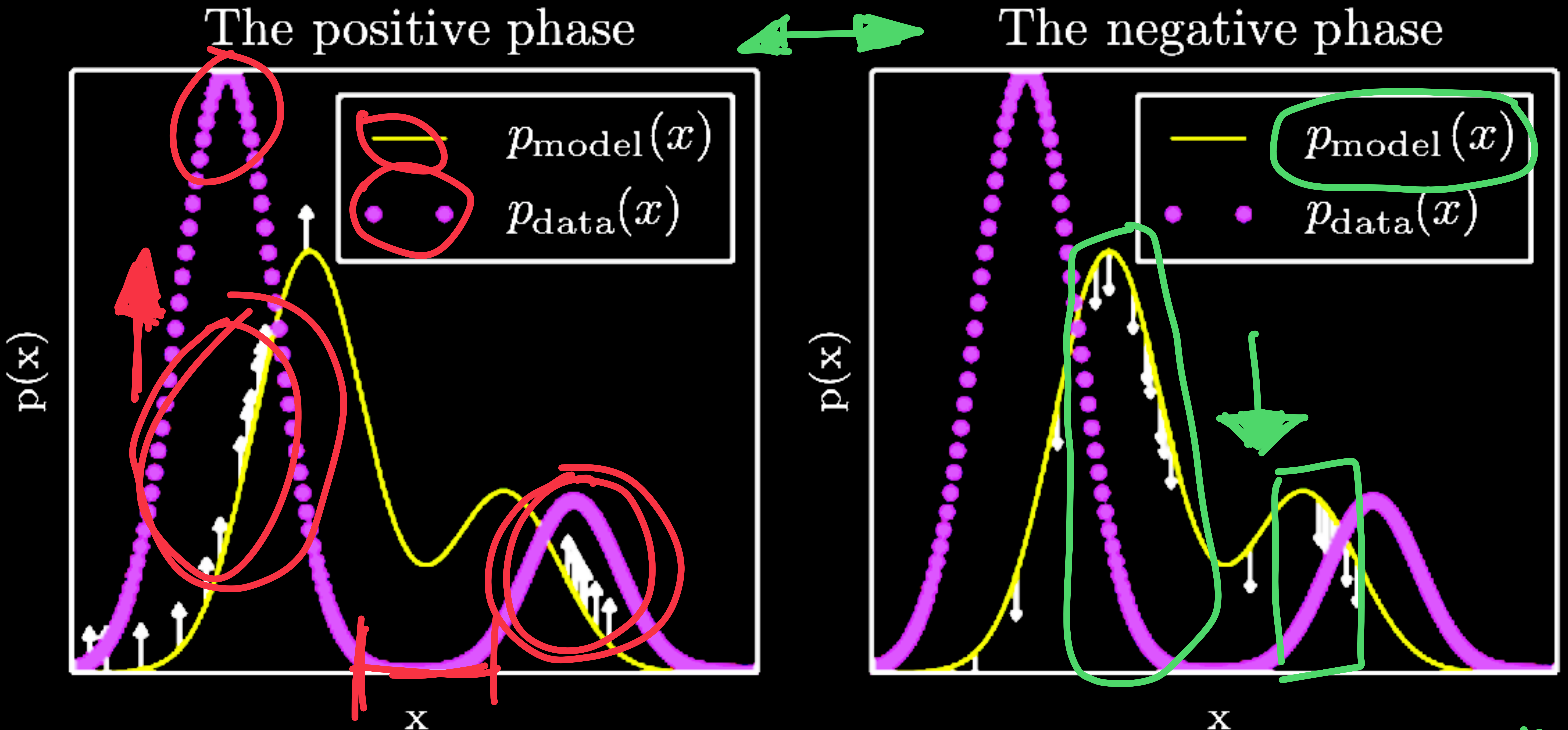
model

model generated data

- Performing gradient ascent using the approximate gradient

$$\boldsymbol{\Theta}^{k+1} = \boldsymbol{\Theta}^{k} + \eta \frac{\partial log(p(\mathbf{x}, \Theta))}{\partial \boldsymbol{\Theta}}\bigg|_{\boldsymbol{\Theta}=\boldsymbol{\Theta}^k}$$

Positive phase and negative phase



Convergence ≈ +ve phase is the same -ve samples

"Motivation"

- Neuroscientific motivation of the learning in the brain
  - Real world samples in our day-to-day [Positive phase] ↑
  - Hallucinations and dreams [Negative phase] suppress ↓

- Learning from the data.
  - Balancing the positive phase based learning with the negative phase.

Gaussian Bernoulli RBM

v - continuous

h - discrete

- For modeling real observations $\mathbf{v} \in \mathcal{R}^D$
- Define the energy function

$$E[\mathbf{v}, \mathbf{h}] = \boxed{\frac{1}{2}(\mathbf{v} - \mathbf{a})^T(\mathbf{v} - \mathbf{a})} - \underline{\mathbf{v}^T\mathbf{W}\mathbf{h}} - \underline{\mathbf{b}^T\mathbf{h}}$$

$$p([\mathbf{v}, \mathbf{h}]) = \frac{e^{-E[\mathbf{v}, \mathbf{h}]}}{Z}$$

Take home exercise.

- The conditional distributions

$$p(\mathbf{v}|\mathbf{h}) = \mathcal{N}(\underline{\mathbf{W}\mathbf{h} + \mathbf{a}}, \mathbf{I})$$

$$p(h_j = 1|\mathbf{v}) = \sigma(\underline{\mathbf{v}^T\mathbf{W}^k_{:,j}} + b_j)$$

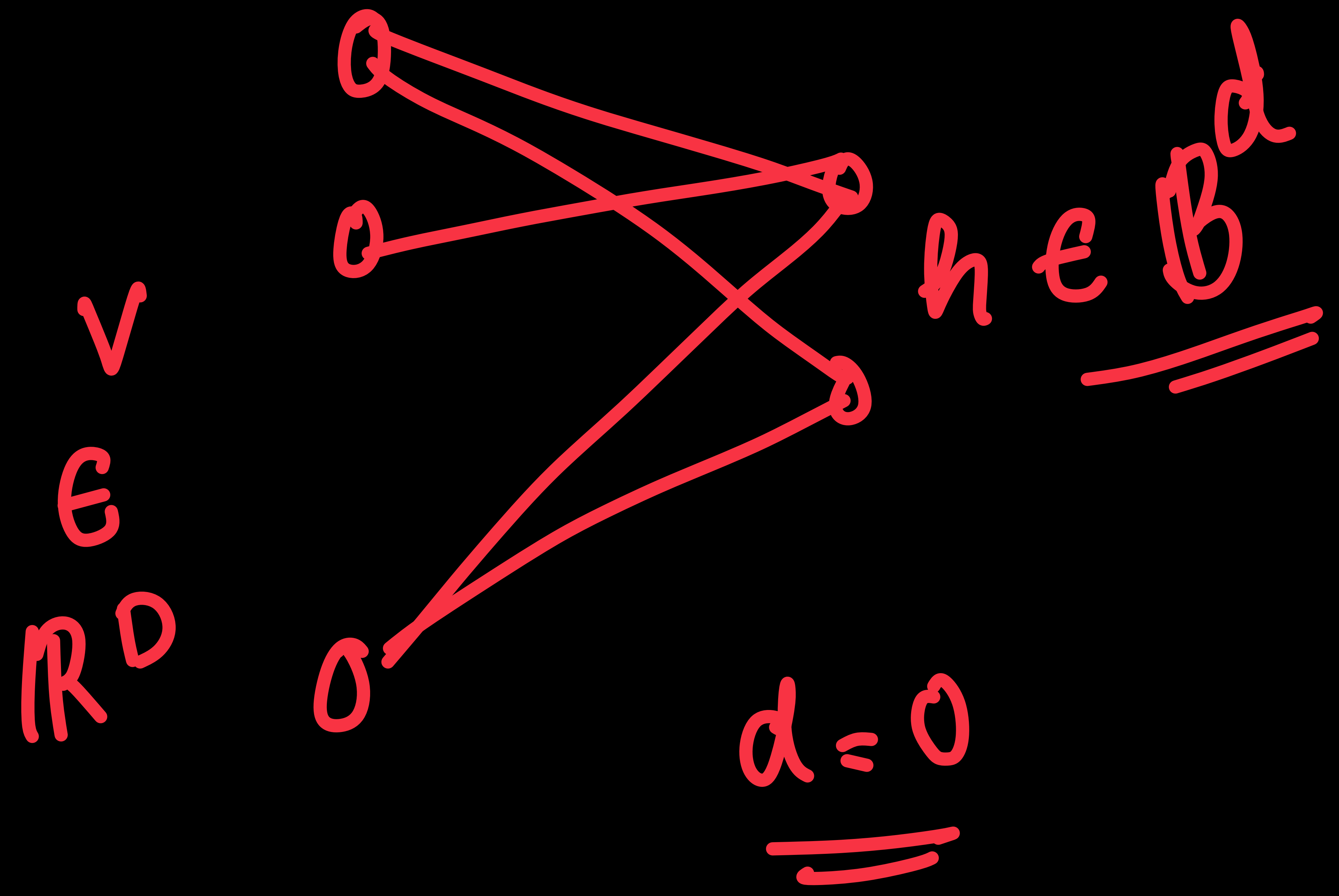
Properties of GRBM

- d = 0

Marginal

$$E(\mathbf{v}) = \frac{1}{2}(\mathbf{v} - \mathbf{a})^T(\mathbf{v} - \mathbf{a})$$

$$\boxed{p(\mathbf{v}) = \frac{e^{-E(\mathbf{v})}}{Z}}$$



- The marginal distribution is a Gaussian. $= N(\underline{a}, I)$

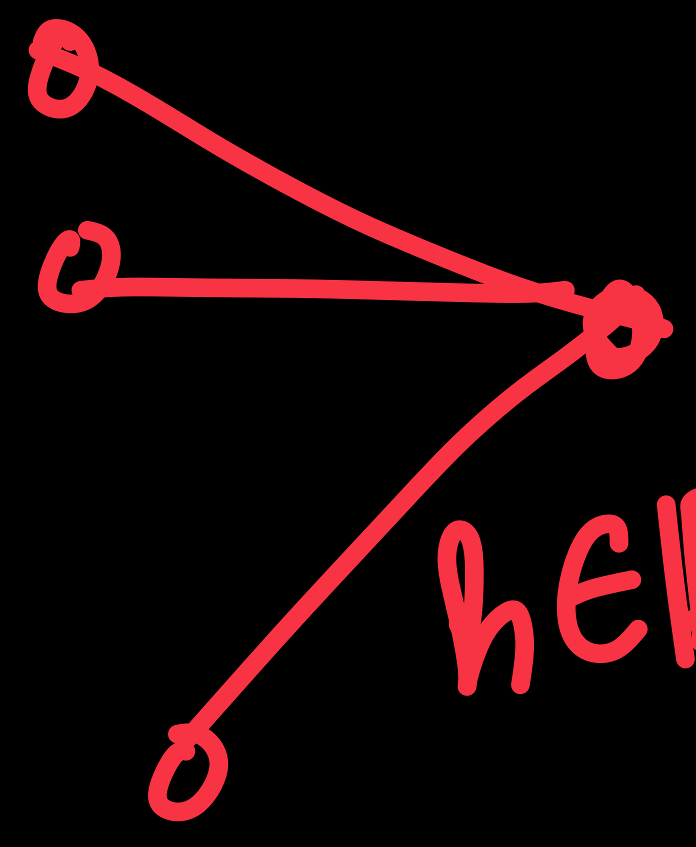
Properties of GRBM

- $\boxed{\text{d = 1}}$

$$E[\mathbf{v}, h] = \frac{1}{2}(\mathbf{v} - \mathbf{a})^T(\mathbf{v} - \mathbf{a}) - h\mathbf{v}^T\mathbf{w} - hb$$

$$p([\mathbf{v}, h]) = \frac{e^{-E[\mathbf{v},h]}}{Z}$$

$\mathbf{v} \in \mathbb{R}^D$



$h \in \mathbb{B}$

$$p([\mathbf{v}, h = 0]) = \alpha\mathcal{N}(\mathbf{a}, \mathbf{I})$$

$$p([\mathbf{v}, h = 1]) = (1 - \alpha)\mathcal{N}(\mathbf{a} + \mathbf{w}, \mathbf{I})$$

$h = 0$

$h = 1$

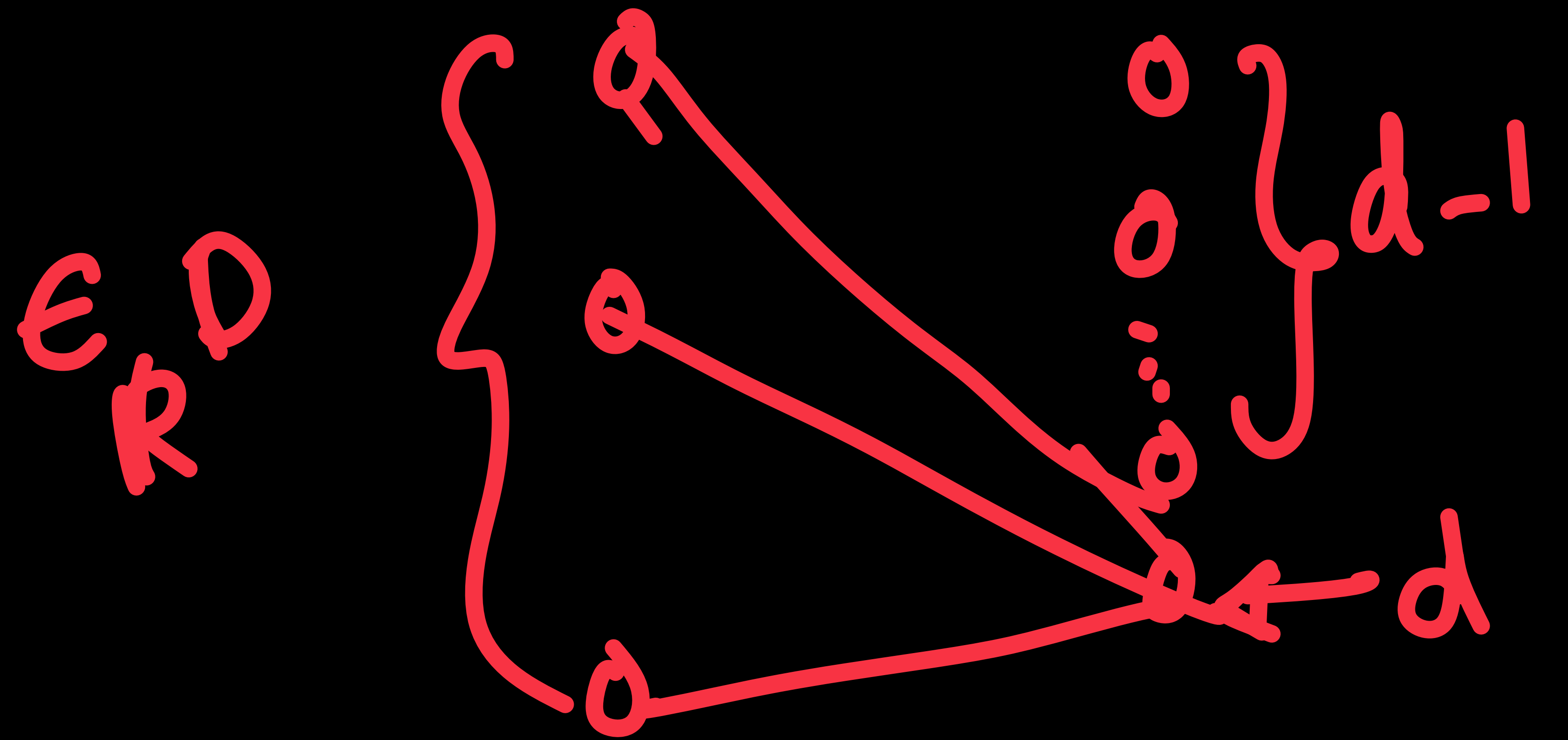
- The marginal distribution is then

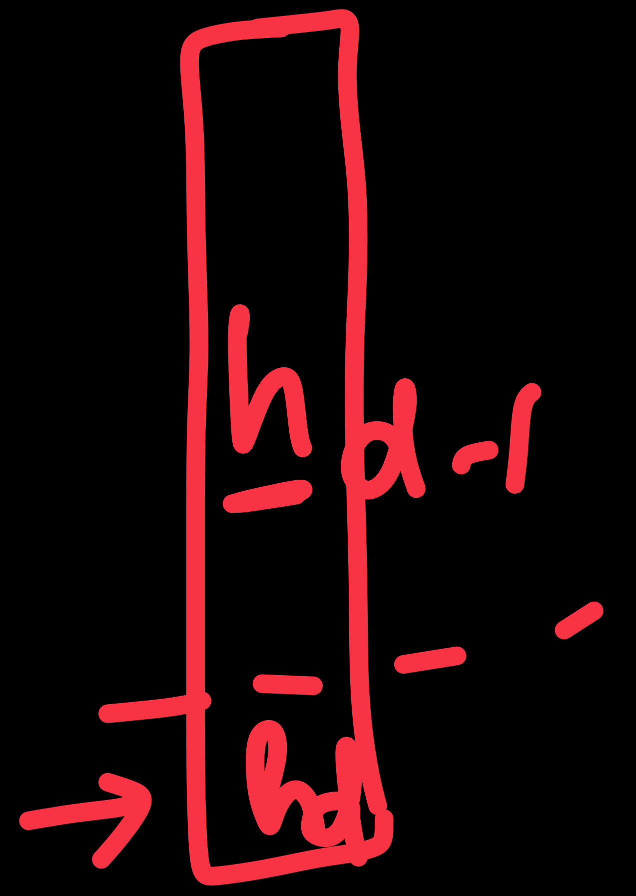
$\alpha N(a, I) + (1-\alpha) N(a+w, I)$

$$p(\mathbf{v}) = p([\mathbf{v}, h = 0]) + p([\mathbf{v}, h = 1])$$

✓ 2-mixture Gaussian

Properties of GRBM





- For any general d dimensions

$$E[\mathbf{v}, \mathbf{h}_d] = E[\mathbf{v}, \mathbf{h}_{d-1}] + h_d \, \mathbf{v}^T \mathbf{W}_{:,d} + b_d h_d$$

$$p([\mathbf{v}, [\mathbf{h}_{d-1}, h_d = 0]]) = \alpha \; p([\mathbf{v}, \mathbf{h}_{d-1}])$$

$$p([\mathbf{v}, [\mathbf{h}_{d-1}, h_d = 1]]) = (1-\alpha) \; p([(\mathbf{v} + \mathbf{W}_{:,d}), \mathbf{h}_{d-1}])$$

$D \times d$

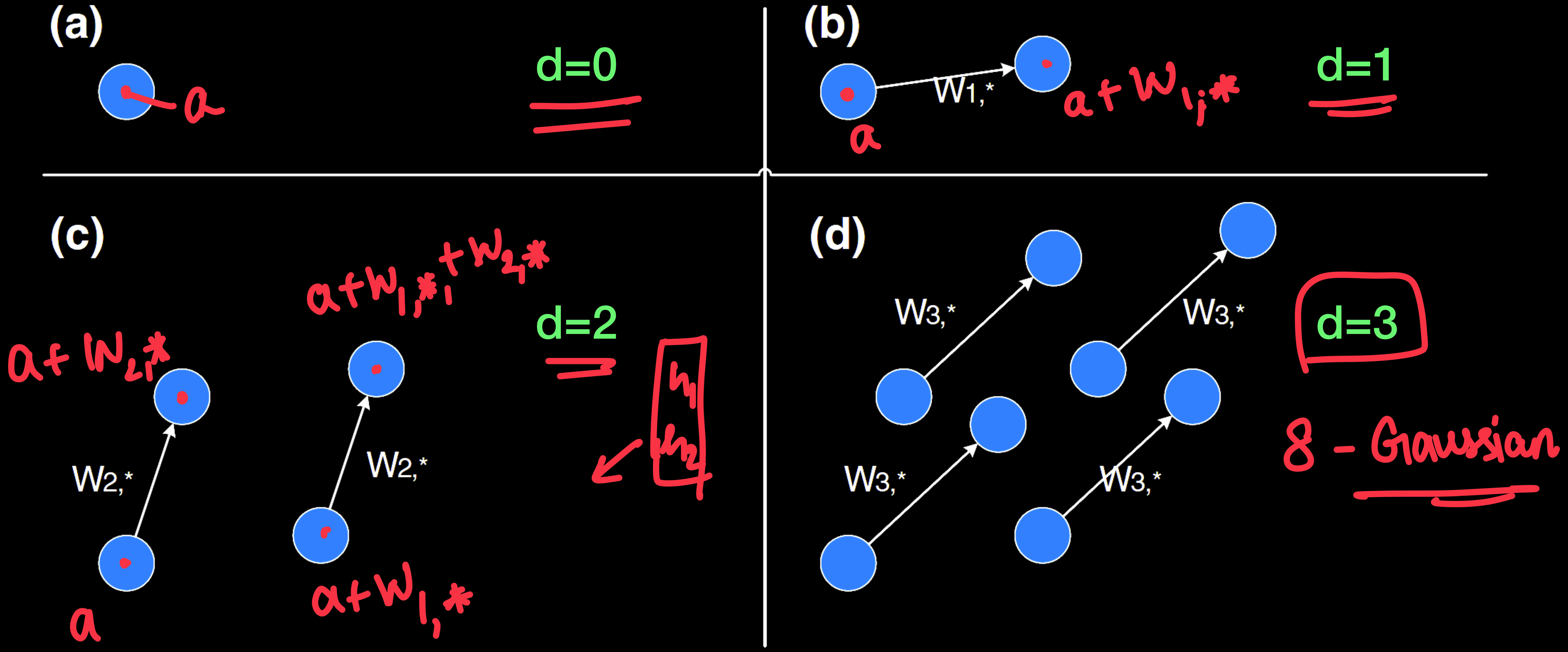
$h_d = \{0, 1\}$

- For d=0, 1 Gaussian, d=1, 2-mix Gaussian, …
  - 2^d mixture Gaussian for any arbitrary d.

$$\mathbf{h}_d = \begin{bmatrix} \mathbf{h}_{d-1} \\ h_d \end{bmatrix}$$

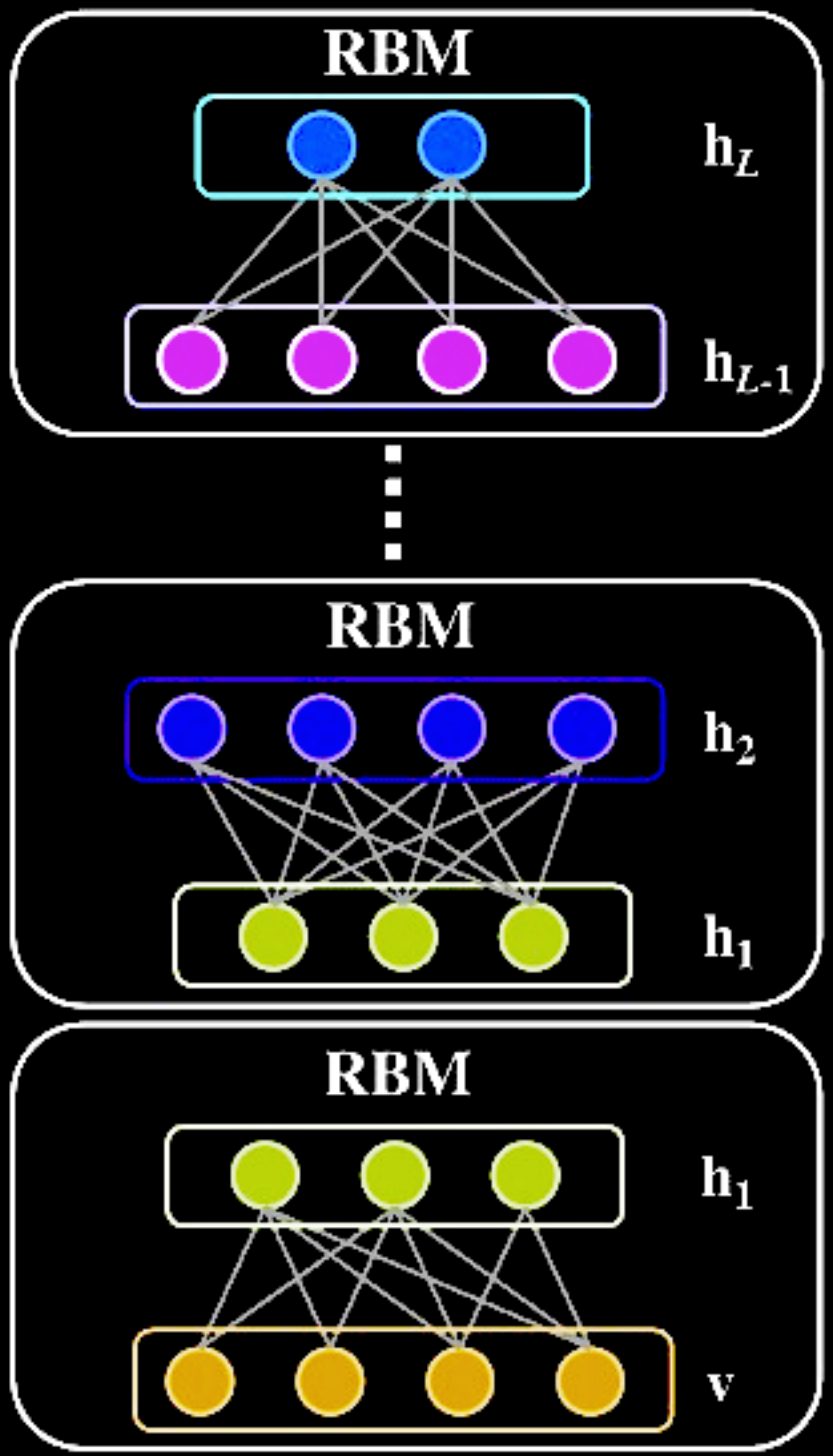
GRBMs and GMMs

$p(v)$

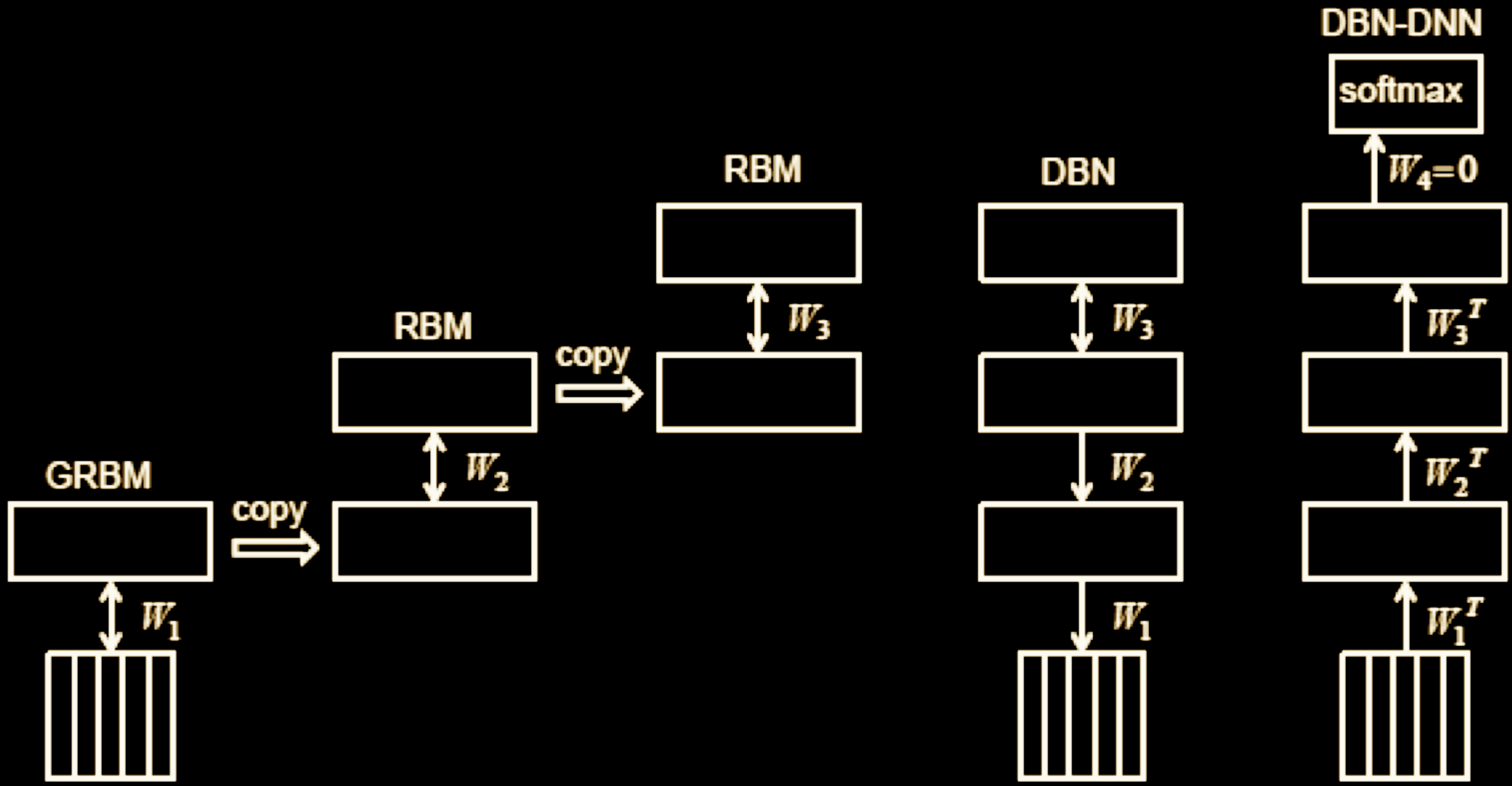


Deep Belief Networks (DBN)

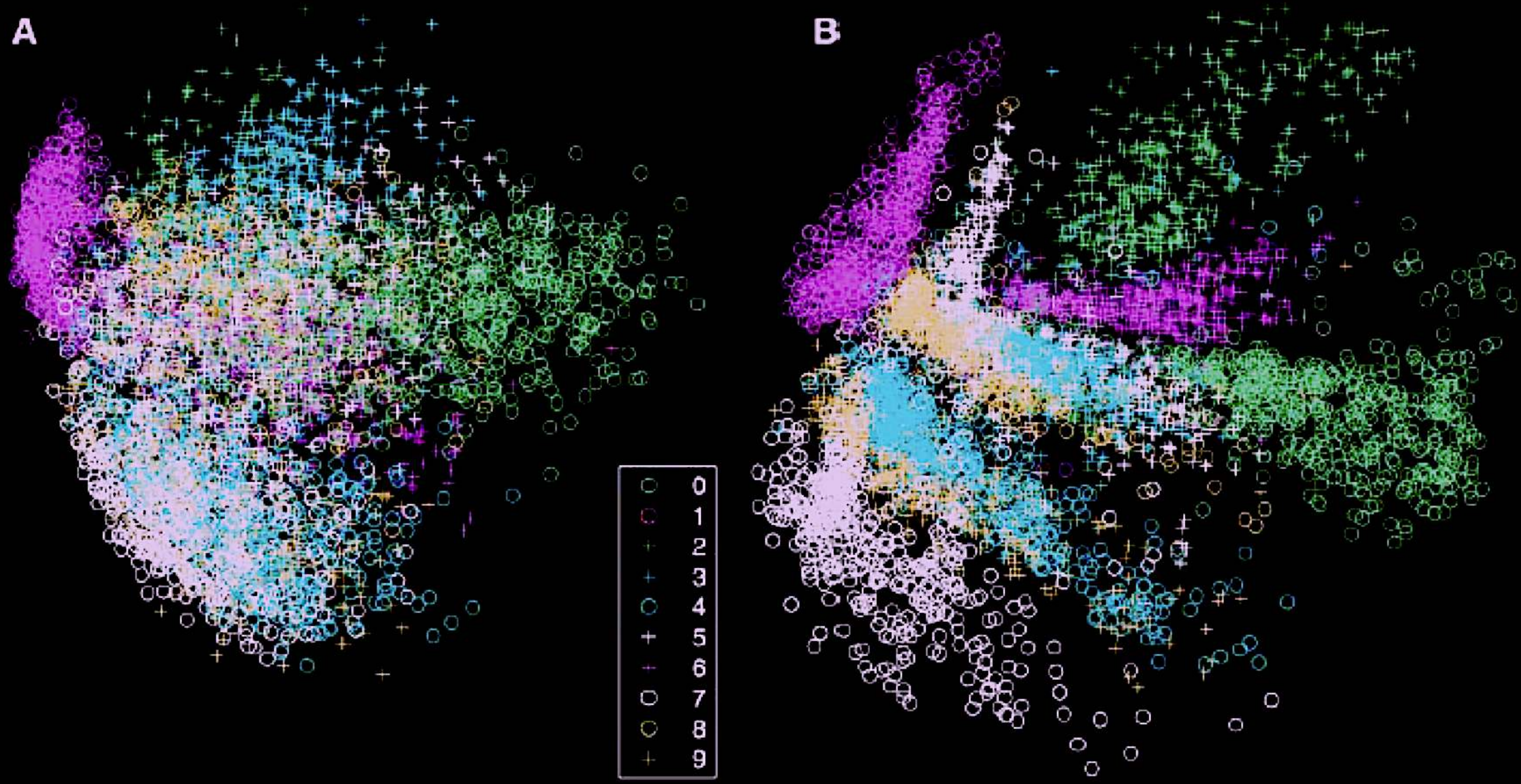
- Stacking RBMs in a disjoint fashion
  - ✓ Layer-wise training for each RBM.
    - ◉ Weights are frozen each layer before training the next layer.
  - ✓ Ancestral sampling can be performed for data generation
    - ★ Lossy sample generation due to accumulation of errors.
- Most common use - pre-training of DNNs.



DBNs for initialization



DBNs for visualization

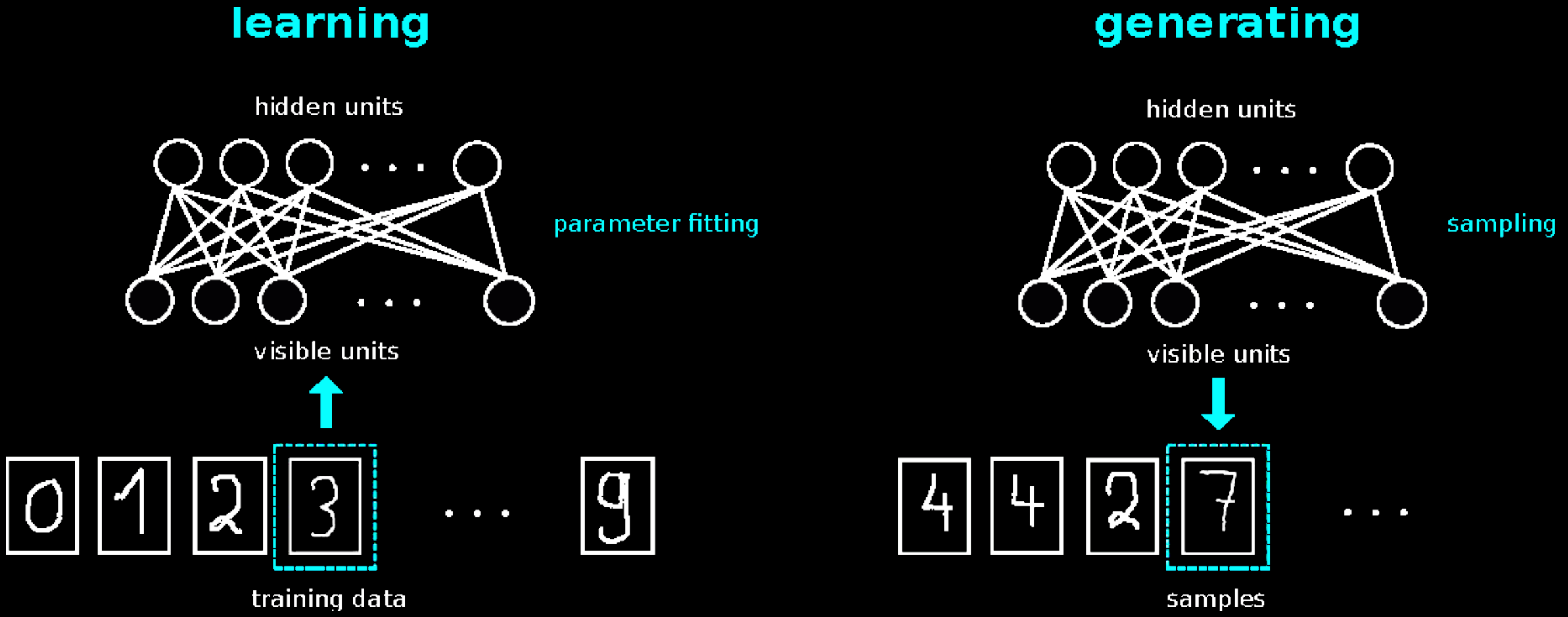


PCA RBM

More reading

Salakhutdinov, Ruslan, Andriy Mnih, and Geoffrey Hinton. "Restricted Boltzmann machines for collaborative filtering." *Proceedings of the 24th international conference on Machine learning*. 2007.

Data generation using RBMs



Data generation using RBMs

Source — https://lme.tf.fau.de/lecture-notes/lecture-notes-in-deep-learning-unsupervised-learning-part-1/

Deep Boltzmann machine

- ✴️ Deep layers of connections with RBM structure.
  - ➡️ Joint energy function.
  - ➡️ Undirected graph